

CSCE 669 COMPUTATIONAL OPTIMIZATION · MODULE 08 · LECTURE NOTES

Stochastic and Large-Scale Methods

When you cannot afford a full gradient.

MODULE OUTCOMES

- State SGD's convergence rate and why it is what it is.
- Explain the noise floor and why the step size must decay.
- Explain variance reduction and when it applies.
- Explain coordinate descent and when it wins.
- Connect the theory to CSCE 636's practice honestly.

1 The trade

SGD TRADES ACCURACY PER STEP FOR STEPS PER SECOND

A full gradient over n training examples costs n times a single example's gradient and produces one parameter update. Every example is consulted before anything moves.

A stochastic gradient costs one example's gradient and also produces one update — so for the same computational budget you take n times as many steps.

Each individual step is worse, and there are vastly more of them. For large n that trade is overwhelmingly favourable in the early phase, where any descent direction is useful and precision is wasted.

But it is unfavourable in the late phase. The gradient noise does not shrink as the iterate approaches the optimum — at x^* the full gradient is zero and the individual gradients are not — so progress stalls at a noise floor (§2). **The method that is best early is not the method that is best late**, which is the whole reason schedules exist.

Setting	Rate per iteration	Rate per <i>gradient evaluation</i>
Gradient descent, strongly convex	Linear — $\log(1/\epsilon)$ iterations (Module 05 §1).	$n \cdot \kappa \log(1/\epsilon)$ — every iteration costs n gradients.
SGD, strongly convex	$O(1/k)$ — sublinear, which looks much worse.	$O(1/\epsilon)$, independent of n . This is the row that matters — the cost to reach a given accuracy does not grow with the dataset at all.
SGD, convex but not strongly	$O(1/\sqrt{k})$.	$O(1/\epsilon^2)$.

Setting	Rate per iteration	Rate per <i>gradient evaluation</i>
SVRG, SAGA (variance-reduced)	Linear.	$(n + \kappa) \log(1/\epsilon)$ — better than both when n and κ are comparable (§3).

Measuring in gradient evaluations rather than in iterations is what makes SGD look good, and it is the honest accounting because gradient evaluations are what cost money. Comparing iteration counts between a full-batch and a stochastic method is meaningless, and it is done surprisingly often.

2 The noise floor

A CONSTANT STEP SIZE CONVERGES TO A BALL, NOT A POINT

With a fixed step size η , SGD converges to a neighbourhood of the optimum whose radius is proportional to η times the gradient variance — and then stays there.

It bounces around inside that ball indefinitely. Additional iterations at the same step size do not improve the solution; they move it around within the same region, which is exactly what a loss curve that has flattened at a disappointing value looks like.

So reduce η to shrink the ball. The classical **Robbins–Monro conditions** are $\sum \eta_k = \infty$ (the steps must sum to infinity, so the iterate can reach anywhere in the space) and $\sum \eta_k^2 < \infty$ (the squares must sum finitely, so the noise is eventually suppressed). A schedule of $1/k$ satisfies both.

This is the theoretical content of CSCE 636 Module 03's learning rate schedules. 'Explore then exploit' is precisely 'traverse the space while the ball is large, then shrink the ball' — and the informal framing there now has a quantity attached to it. Decaying too early traps you with a small ball far from the optimum; never decaying leaves you orbiting it forever.

LARGER BATCHES SHRINK THE NOISE, AT A COST

Gradient variance falls as $1/B$ with batch size B — averaging B independent estimates — so the noise ball of the previous callout shrinks as the batch grows.

But the cost per step grows linearly in B , so the trade is not free: doubling the batch halves the variance and doubles the work per step. Past some point the variance is no longer what limits progress, and further batch growth buys nothing.

That point is the **critical batch size**, it is measurable for a given problem, and beyond it you are paying linearly for steps that are not meaningfully better — which is the regime very large distributed training runs operate in, and why their scaling efficiency falls off.

And the linear scaling rule — double the batch, double the learning rate — holds approximately up to that point and fails beyond it. Which is why very large batch training requires warmup and layer-wise rate scaling (CSCE 636 Module 08 §2): the naive rule puts the step size outside the stable range that Module 05 §1's smoothness constant permits.

3 Variance reduction

THE PROBLEM: SGD's gradient variance does not vanish near the optimum, so it cannot converge linearly.

SVRG: occasionally compute a FULL gradient at a snapshot $\tilde{x}$, then correct each stochastic gradient with it:

$$g = \text{grad } f_i(x) - \underbrace{\text{grad } f_i(\tilde{x})}_{\text{control variate}} + \text{grad } F(\tilde{x})$$

UNBIASED -- the correction has expectation zero -- and its VARIANCE SHRINKS as x approaches $\tilde{x}$ and both approach x^* .

RESULT: linear convergence, at $(n + \kappa) \log(1/\epsilon)$ gradient evaluations. Better than GD and SGD when n and κ are comparable.

WHERE IT APPLIES

finite sums over a FIXED dataset -- exactly the empirical risk minimisation of CSCE 633.

WHERE IT DOES NOT

streaming or infinite data (no full gradient exists), and DEEP LEARNING, where it reliably underperforms plain SGD for reasons still debated.

It is a beautiful result that does not help deep learning, and saying so is better than implying otherwise. The leading explanations are that the snapshot goes stale too quickly when the iterate moves far, that data augmentation makes the 'fixed finite sum' assumption false, and that **SGD's noise may be beneficial rather than merely tolerated** (CSCE 636 Module 03 §2) — in which case reducing it is counterproductive. **None is settled**, and the honest summary is that the method works where its assumptions hold and the assumptions do not hold in the setting that currently matters most.

4 Coordinate descent

COORDINATE DESCENT WINS WHEN A SINGLE COORDINATE IS CHEAP

Update one coordinate at a time, exactly, holding all the others fixed, and cycle (or choose randomly). Each update solves a one-dimensional problem.

For the lasso, each coordinate update has a closed form — soft thresholding again (Module 05 §3) — and costs $O(n)$ rather than the $O(np)$ of a full gradient step. **With active-set tracking, most coordinates are zero and are skipped entirely**, which is where the real speed comes from.

So it is the standard method for the lasso and elastic net, and it is what glmnet and scikit-learn's coordinate descent solver implement — it is why fitting a full regularisation path is so fast (CSCE 633 Module 04 §1).

It handles non-smooth separable penalties naturally, which is exactly the ℓ_1 case: the penalty decomposes across coordinates, so a one-dimensional subproblem is well posed. **And it fails on non-separable penalties** — the fused lasso, group lasso with overlaps — **because no single coordinate can move without violating a coupling**, so the method stalls at a non-optimal point. **Separability is the precondition, and checking it is the first thing to do.**

- **Huge n , moderate accuracy sufficient: SGD** with a decaying schedule satisfying §2's conditions.
- **Fixed finite sum, high accuracy required, convex: SVRG or SAGA** — where the linear rate genuinely pays.
- **Separable non-smooth penalty: coordinate descent** (§4).
- **Composite objective with exploitable structure: ADMM** (Module 06 §3).
- **And deep learning: SGD with momentum, or AdamW** — where this course's theory offers guidance rather than guarantees. **The gap between the convex theory and deep learning practice is real**, and it is narrowest at the learning-rate schedule (§2) and widest at the question of why the resulting solutions generalise (CSCE 636 Module 03 §4).

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Bottou, Curtis & Nocedal — Optimization Methods for Large-Scale Machine Learning (free)	The reference for this whole module. The clearest treatment of the §1 trade and the §2 noise analysis.
Johnson & Zhang — Accelerating Stochastic Gradient Descent using Predictive Variance Reduction (SVRG) (free)	The §3 method, with the control variate argument.

Resource	What it covers for this module
Goyal et al. — Accurate, Large Minibatch SGD (free)	The linear scaling rule of §2 , its warmup requirement, and where it breaks.
Friedman, Hastie & Tibshirani — Regularization Paths for Generalized Linear Models via Coordinate Descent (free)	The §4 method as implemented in <code>glmnet</code> , including the active set and path-following tricks.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Compare GD and SGD on the same problem**, plotting error against gradient evaluations rather than iterations.
- 2 **Run SGD with a constant step size** and plot the iterate's distance from the optimum. Show it plateaus.
- 3 **Vary the step size and show the plateau radius scales with it.**
- 4 Add a $1/k$ schedule and show convergence to the optimum.
- 5 **Measure gradient variance against batch size** and confirm the $1/B$ relationship.
- 6 **Find the critical batch size** for your problem by plotting steps to a target loss against batch size.
- 7 Implement SVRG and compare against SGD on a convex finite sum.
- 8 **Then try SVRG on a small neural network** and report what happens.
- 9 **Implement coordinate descent for the lasso** and compare against proximal gradient.
- 10 Apply coordinate descent to a fused lasso and show it stalls.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What does SGD trade, and why is the gradient-evaluation column the right one?
- 2 Why does SGD's noise not vanish near the optimum?
- 3 What does a constant step size converge to, and what are the Robbins–Monro conditions?
- 4 How does batch size affect variance and cost, and what is the critical batch size?
- 5 Explain SVRG's control variate and the rate it achieves.
- 6 Where does SVRG not apply, and what are the explanations?
- 7 When does coordinate descent win, and what precondition does it need?